

Functional Requirements Document (FRD)
Algorithmic Trading System

Document Control

Document Title: Functional Requirements Document for Algorithmic Trading System

Version: 1.0

Date: April 24, 2025

Author: Senior Systems Developer, Hedge Fund

Status: Draft

Distribution: Development Team, Product Owner, Traders, Compliance Team, Stakeholders

Revision History

Version	Date	Author	Description
1.0	04/24/2025	Senior Systems Developer	Initial draft of FRD for Algorithmic Trading System

Approval

Role	Name	Signature	Date
Product Owner	[To be assigned]		
Head of Trading	[To be assigned]		
Compliance Officer	[To be assigned]		
Lead Developer	[To be assigned]		

Table of Contents

- 1. [Introduction](#)
 - 1.1. Purpose
 - 1.2. Scope
 - 1.3. Objectives
 - 1.4. Assumptions and Constraints
 - 1.5. Stakeholders
 - 1.6. Definitions and Acronyms
- 2. [System Overview](#)
 - 2.1. System Description
 - 2.2. System Context
 - 2.3. Key Features
- 3. [Functional Requirements](#)
 - 3.1. Trading Strategies
 - 3.1.1. Trend-Following Strategy
 - 3.1.2. Support and Resistance Strategy
 - 3.2. Market Watch Dashboard
 - 3.2.1. Scrip Selection
 - 3.2.2. Real-Time Data Display
 - 3.2.3. Interactive Charts
 - 3.2.4. Portfolio Summary
 - 3.2.5. Simulated Trading Panel
 - 3.2.6. Watchlist
 - 3.2.7. Technical Analysis
 - 3.2.8. News and Events
 - 3.3. Data Management
 - 3.3.1. Data Ingestion
 - 3.3.2. Data Processing
 - 3.3.3. Data Storage
 - 3.3.4. Data Visualization
 - 3.4. Order Execution
 - 3.4.1. Order Types
 - 3.4.2. Order Placement
 - 3.4.3. Order Monitoring

- 3.4.4. Error Handling
- 3.5. Risk Management
 - 3.5.1. Position Sizing
 - 3.5.2. Stop-Loss and Take-Profit
 - 3.5.3. Trailing Stop
 - 3.5.4. Portfolio Risk
 - 3.5.5. Drawdown Limits
 - 3.5.6. Circuit Breakers
 - 3.5.7. Currency Handling
- 3.6. AI Agents Integration
 - 3.6.1. Trade Execution Agent
 - 3.6.2. Risk Management Agent
 - 3.6.3. Market Analysis Agent
 - 3.6.4. IBKR API Agent
 - 3.6.5. Transparency and Oversight
- 3.7. User Interface
 - 3.7.1. Frontend Components
 - 3.7.2. Interactivity
 - 3.7.3. Responsiveness
 - 3.7.4. Accessibility
- 3.8. Reporting and Analytics
 - 3.8.1. Trade Performance
 - 3.8.2. Portfolio Analytics
 - 3.8.3. Backtesting Reports
 - 3.8.4. Compliance Reports
 - 3.8.5. Export Capabilities
- 4. [Non-Functional Requirements](#)
 - 4.1. Performance
 - 4.2. Scalability
 - 4.3. Security
 - 4.4. Reliability and Fault Tolerance
 - 4.5. Usability
 - 4.6. Compliance
- 5. [Integration Requirements](#)
 - 5.1. Interactive Brokers Paper Trading API
 - 5.2. External Data Sources
 - 5.3. Internal Systems
- 6. [Assumptions and Dependencies](#)
 - 6.1. Assumptions Log
 - 6.2. Dependencies
- 7. [Missing Information and Best Practices](#)
 - 7.1. Missing Information
 - 7.2. Best Practices Applied
- 8. [Glossary](#)
- 9. [Appendices](#)
 - 9.1. References
 - 9.2. Sample Data Formats
 - 9.3. Mockups and Wireframes

1. Introduction

1.1. Purpose

The Functional Requirements Document (FRD) for the Algorithmic Trading System (ATS) defines the detailed functional specifications required to develop a robust, high-performance platform for simulated trading. This document serves as a comprehensive guide for the development team, product owner, traders, and compliance officers to ensure alignment on the system's capabilities, behavior, and deliverables. It translates business needs into actionable technical requirements, specifying how the system will operate,

interact with users, and integrate with external systems. The FRD ensures that all functional aspects of the ATS, from trading strategies to user interfaces, are clearly defined to support the hedge fund's strategic objectives.

1.2. Scope

The ATS is a sophisticated platform designed to execute paper trades across multiple asset classes, including stocks, indices, forex, and commodities, on major global exchanges such as NYSE, NASDAQ, LSE, and NSE. The system integrates with the Interactive Brokers (IBKR) Paper Trading API for simulated trading and leverages advanced data processing libraries (Polars, DuckDB) and AI agents for enhanced decision-making. The scope includes:

- **Trading Strategies:** Implementation of Trend-Following and Support and Resistance strategies for long and short positions.
- **Market Watch Dashboard:** A responsive web interface for real-time market monitoring, technical analysis, portfolio management, and simulated trading.
- **Data Management:** Efficient ingestion, processing, storage, and visualization of market data.
- **Order Execution:** Placement and monitoring of simulated trades via the IBKR API.
- **Risk Management:** Position sizing, stop-loss, take-profit, and portfolio-level risk controls.
- **AI Agents:** Modular components for market analysis, trade execution, and risk assessment.
- **User Interface:** An intuitive, responsive frontend for traders.
- **Reporting and Analytics:** Trade performance metrics, backtesting results, and compliance reports.

Out of Scope:

- Live trading functionality.
- Integration with brokers other than IBKR.
- Support for cryptocurrencies or non-traditional assets.
- Mobile app development (dashboard will be mobile-responsive).
- Advanced AI model training (pre-trained models will be used).

1.3. Objectives

The ATS aims to achieve the following functional objectives:

1. **Automate Trading:** Execute algorithmic trading strategies with minimal manual intervention.
2. **Provide Real-Time Insights:** Deliver live market data, technical indicators, and AI-driven recommendations to traders.
3. **Ensure Robust Risk Management:** Implement controls to limit financial exposure and protect capital.
4. **Facilitate Strategy Testing:** Support backtesting and paper trading to validate strategies.
5. **Enhance User Experience:** Offer an intuitive, responsive dashboard for monitoring and trading.
6. **Support Compliance:** Generate audit trails and reports to meet regulatory requirements.
7. **Enable Scalability:** Build a modular system to accommodate future enhancements.

1.4. Assumptions and Constraints

Assumptions:

- The IBKR Paper Trading API provides reliable, timely market data and order execution.
- Traders have intermediate knowledge of algorithmic trading and technical analysis.
- The Lenovo Legion Pro 5 (NVIDIA RTX 3060 GPU) supports system performance requirements.
- Regulatory requirements for paper trading are minimal compared to live trading.
- The development team is proficient in Python, Polars, DuckDB, React, and FastAPI.

Constraints:

- Development timeline: 3-6 months to align with strategic goals.
- IBKR API rate limits: 50 requests/second for market data, 100 messages/second for orders.
- Budget limits: No premium third-party data sources or cloud hosting initially.
- Deployment: Local on Lenovo Legion Pro 5, no cloud infrastructure.
- Compliance: Must adhere to SEC, GDPR, MiFID II, and SEBI regulations.

1.5. Stakeholders

Stakeholder	Role	Responsibilities
Head of Trading	Oversees trading operations	Defines strategy requirements, validates functionality
Traders	End-users	Test system, provide usability feedback

Stakeholder	Role	Responsibilities
Product Owner	Represents business	Prioritizes features, approves deliverables
Development Team	Builds and tests system	Implements requirements, ensures quality
Compliance Officer	Ensures regulatory adherence	Reviews system for compliance
IT Administrator	Manages infrastructure	Configures hardware, ensures uptime

1.6. Definitions and Acronyms

- **ATS:** Algorithmic Trading System.
- **IBKR:** Interactive Brokers.
- **OHLCV:** Open, High, Low, Close, Volume.
- **VW SMA:** Volume-Weighted Simple Moving Average.
- **MFI:** Money Flow Index.
- **ATR:** Average True Range.
- **Polars:** High-performance data processing library.
- **DuckDB:** In-memory analytical database.
- **FastAPI:** Python framework for APIs.
- **ib_insync:** Python library for IBKR API integration.

2. System Overview

2.1. System Description

The ATS is a comprehensive platform for algorithmic paper trading, enabling simulated trades across stocks, indices, forex, and commodities. It integrates with the IBKR Paper Trading API for data and execution, using Polars and DuckDB for data processing and storage. The system features two trading strategies (Trend-Following and Support and Resistance), a Market Watch Dashboard, AI agents, and robust risk management. Deployed locally on the Lenovo Legion Pro 5, the ATS uses a modular, event-driven architecture to ensure performance, scalability, and maintainability.

2.2. System Context

The ATS operates within the following context:

- **External Systems:**
 - IBKR Paper Trading API: Provides market data and trade execution.
 - Future data sources (out of scope): News feeds, sentiment data.
- **Internal Systems:**
 - Portfolio management tools for benchmarking.
 - Compliance systems for transaction logging.
- **Users:**
 - Traders: Interact via the Market Watch Dashboard.
 - Administrators: Manage system configuration and compliance.
- **Environment:**
 - Local deployment with Docker containerization.
 - Event-driven workflows for real-time processing.

2.3. Key Features

- **Multi-Asset Trading:** Supports stocks, indices, forex, and commodities on major exchanges.
- **Real-Time Monitoring:** Displays live prices, volume, and technical indicators.
- **Automated Strategies:** Executes Trend-Following and Support and Resistance strategies.
- **Interactive Dashboard:** Offers charts, portfolio summaries, and trading controls.
- **AI Insights:** Automates analysis, execution, and risk management.
- **Backtesting and Simulation:** Validates strategies using historical and paper trading data.
- **Security:** Ensures encrypted communications and secure credential management.
- **Reporting:** Provides trade performance, portfolio analytics, and compliance reports.

3. Functional Requirements

3.1. Trading Strategies

The ATS implements two positional trading strategies: Trend-Following and Support and Resistance, supporting long and short positions across multiple asset classes in the IBKR paper trading environment.

3.1.1. Trend-Following Strategy

Description: A strategy that captures long-term price trends using volume-weighted technical indicators to assess direction, momentum, and volatility.

Functional Requirements:

- **Assets:**
 - Stocks: NYSE (e.g., AAPL, MSFT), NASDAQ (e.g., TSLA), LSE (e.g., BP), NSE (e.g., RELIANCE).
 - Indices: S&P 500, FTSE 100, NIFTY 50.
 - Forex: EUR/USD, USD/JPY, GBP/USD.
 - Commodities: Gold (XAU/USD), Crude Oil (WTI).
- **Technical Indicators:**
 - **55-day VW SMA of Open:** Identifies long-term trend.
 - Formula:
$$\left[\frac{\sum (\text{Open Price}_i \times \text{Volume}_i)}{\sum \text{Volume}_i} \right]$$
where (i) spans 55 days.
 - **13-day VW SMA of Open:** Tracks medium-term trends (optional).
 - **14-day VW MFI of HLC Average:** Measures momentum.
 - $\text{HLC Average} = (\text{High} + \text{Low} + \text{Close}) / 3.$
 - $\text{Raw Money Flow} = \text{HLC Average} \times \text{Volume}.$
 - $\text{Money Flow Ratio} = (\text{14-day Positive Money Flow}) / (\text{14-day Negative Money Flow}).$
 - $\text{MFI} = 100 - (100 / (1 + \text{Money Flow Ratio})).$
 - **34-day VW SMA of MFI(14):** Smooths MFI for trend confirmation.
 - **21-day VW SMA of MFI(14):** Triggers exits.
 - **5-day VW SMA of High:** Defines short-term resistance.
 - **5-day VW SMA of Low:** Defines short-term support.
 - **21-day VW ATR:** Measures volatility.
 - $\text{True Range} = \max(\text{High} - \text{Low}, |\text{High} - \text{Previous Close}|, |\text{Low} - \text{Previous Close}|).$
 - $\text{ATR} = \text{VW SMA of True Range over 21 days}.$
- **Entry Rules:**
 - **Long Position:**
 - 14-day VW MFI > 34-day VW SMA of MFI(14).
 - Current price > 55-day VW SMA of Open.
 - Action: Submit buy order via IBKR API with calculated position size.
 - **Short Position:**
 - 14-day VW MFI < 34-day VW SMA of MFI(14).
 - Current price < 55-day VW SMA of Open.
 - Action: Submit sell order via IBKR API with calculated position size.
- **Exit Rules:**
 - **Long Position:**
 - Stop-Loss: Entry price - (2 × 21-day VW ATR).
 - Take-Profit: Entry price + (2 × 21-day VW ATR).
 - Additional Exits:
 - 14-day VW MFI < 21-day VW SMA of MFI(14).
 - Current price < 5-day VW SMA of Low.
 - **Short Position:**
 - Stop-Loss: Entry price + (2 × 21-day VW ATR).
 - Take-Profit: Entry price - (2 × 21-day VW ATR).
 - Additional Exits:
 - 14-day VW MFI > 21-day VW SMA of MFI(14).
 - Current price > 5-day VW SMA of High.
- **Position Sizing:**
 - Risk per trade: 1% of account balance (£10,000 initial).

- Formula:

$$\left[\text{Position Size} = \frac{\text{Account Balance} \times 0.01}{2 \times \text{21-day VW ATR} \times \text{Lot Size}} \right]$$
- Lot Size:
 - Stocks: 1 share.
 - Forex: 100,000 units.
 - Commodities: Per contract (e.g., 100 oz for gold).
 - Indices: Per contract multiplier (e.g., 10 for S&P 500).
- Round to nearest tradable unit.
- **Money Management:**
 - **Maximum Drawdown:** If drawdown > 20%, reduce risk to 0.5% per trade.
 - **Trailing Stop:** $1.5 \times 21\text{-day VW ATR}$ below highest price (long) or above lowest price (short).
 - **Currency Adjustments:** Convert risk to GBP using real-time exchange rates from IBKR API.
- **Implementation:**
 - Use Polars to compute indicators in parallel for multiple scrips.
 - Trigger trades via event-driven architecture using RabbitMQ.
 - Log all trade signals and executions for auditing.

3.1.2. Support and Resistance Strategy

Description: A strategy that trades breakouts and reversals at key support and resistance levels using Pivot Points, Fibonacci retracement, and Gann Fan.

Functional Requirements:

- **Assets:** Same as Trend-Following Strategy.
- **Technical Tools:**
 - **Pivot Points** (daily):
 - Pivot Point (PP) = $(\text{High} + \text{Low} + \text{Close}) / 3$.
 - Resistance 1 (R1) = $2 \times \text{PP} - \text{Low}$.
 - Support 1 (S1) = $2 \times \text{PP} - \text{High}$.
 - Resistance 2 (R2) = $\text{PP} + (\text{High} - \text{Low})$.
 - Support 2 (S2) = $\text{PP} - (\text{High} - \text{Low})$.
 - **Fibonacci Retracement:** Levels at 23.6%, 38.2%, 50%, 61.8%, 100% based on recent swing high/low.
 - **Gann Fan:** Angles (1x1, 1x2, 2x1) from significant highs/lows, scaled to price and time.
- **Entry Rules:**
 - **Long Position:**
 - Price breaks above R1 or a Fibonacci level (e.g., 61.8%) with rising volume.
 - Price bounces off a Gann Fan angle (e.g., 1x1) in an uptrend.
 - Optional: Confirm with bullish candlestick (e.g., engulfing, hammer).
 - Action: Submit buy order via IBKR API.
 - **Short Position:**
 - Price breaks below S1 or a Fibonacci level (e.g., 38.2%) with rising volume.
 - Price rejected at a Gann Fan angle (e.g., 1x1) in a downtrend.
 - Optional: Confirm with bearish candlestick (e.g., shooting star, bearish engulfing).
 - Action: Submit sell order via IBKR API.
- **Exit Rules:**
 - **Long Position:**
 - Take-Profit: At R2, next Fibonacci level, or next Gann Fan angle.
 - Stop-Loss: Below S1, recent swing low, or 1-2% below entry.
 - **Short Position:**
 - Take-Profit: At S2, next Fibonacci level, or next Gann Fan angle.
 - Stop-Loss: Above R1, recent swing high, or 1-2% above entry.
 - **Trailing Stop:**
 - Percentage-Based: 2-3% below highest price (long) or above lowest price (short).

- Volatility-Based: $2 \times 14\text{-day ATR}$.
- **Position Sizing:**
 - Risk per trade: 1-2% of account balance.
 - Formula:

$$\left[\frac{\text{Account Balance} \times \text{Risk \%}}{|\text{Entry Price} - \text{Stop Loss Price}|} \right]$$
 - Adjust for lot size and round to nearest tradable unit.
- **Money Management:**
 - Total portfolio risk: <5-10% across all trades.
 - Diversification: Limit exposure to single asset or sector.
 - Volatility Adjustment: Scale position sizes using 14-day ATR.
- **Implementation:**
 - Compute Pivot Points and Fibonacci levels daily using Polars.
 - Plot Gann Fan angles dynamically with Plotly.js.
 - Validate signals with volume and candlestick patterns.

3.2. Market Watch Dashboard

Description: A web-based, responsive dashboard for real-time market monitoring, technical analysis, portfolio management, and simulated trading.

3.2.1. Scrip Selection

- **Functionality:**
 - Provide a dropdown or searchable input field to select scrips by ticker (e.g., AAPL, EUR/USD).
 - Support multiple exchanges (NYSE, NASDAQ, LSE, NSE).
 - Auto-suggest tickers based on user input.
- **Validation:**
 - Verify ticker validity via IBKR API.
 - Display error if ticker is invalid or unsupported.
- **Implementation:**
 - React component with asynchronous API calls to IBKR for ticker data.
 - Cache recent selections in local storage.

3.2.2. Real-Time Data Display

- **Functionality:**
 - Display bid, ask, last price, volume, and OHLCV in a table or ticker format.
 - Update every 5-10 seconds via IBKR WebSocket.
 - Highlight price changes (green for up, red for down).
- **Data Fields:**
 - Ticker, exchange, bid, ask, last price, volume, open, high, low, close.
 - Percentage change (daily, weekly).
- **Implementation:**
 - Use WebSocket for real-time updates.
 - Store data in Redux for state management.

3.2.3. Interactive Charts

- **Functionality:**
 - Render candlestick or line charts with zoom/pan capabilities.
 - Overlay indicators: VW SMA, MFI, ATR, RSI(14), MACD, Pivot Points, Fibonacci, Gann Fan.
 - Support timeframes: 1m, 5m, 15m, 1h, 4h, 1d, 1w.
 - Allow toggling indicators on/off.
- **Features:**
 - Crosshair for price/time details.
 - Export charts as PNG.
 - Save chart settings per user.
- **Implementation:**
 - Plotly.js for charting.

- Polars for indicator calculations, served via FastAPI.

3.2.4. Portfolio Summary

- **Functionality:**
 - Display open positions, account value, unrealized P&L.
 - Show cash balance, margin, and equity.
 - Filter by asset type, exchange, or strategy.
- **Fields:**
 - Ticker, position size, entry price, current price, P&L, holding period.
 - Total account value, daily/weekly P&L.
- **Implementation:**
 - React table component with sorting and filtering.
 - Fetch data from IBKR API via FastAPI.

3.2.5. Simulated Trading Panel

- **Functionality:**
 - Place buy/sell orders (market, limit, stop, bracket).
 - Input fields: ticker, quantity, price, order type.
 - Validate inputs (e.g., sufficient funds, valid price).
 - Display order status (pending, filled, canceled).
- **Features:**
 - Preview order impact on portfolio.
 - Cancel or modify open orders.
- **Implementation:**
 - React form with `ib_insync` integration.
 - Real-time status updates via WebSocket.

3.2.6. Watchlist

- **Functionality:**
 - Save and manage favorite scrips.
 - Highlight scrips with high volatility (e.g., $ATR > \text{threshold}$) or volume.
 - Allow reordering and grouping by asset type.
- **Features:**
 - Add/remove scrips with one click.
 - Persist watchlist per user.
- **Implementation:**
 - React component with local storage or backend persistence.

3.2.7. Technical Analysis

- **Functionality:**
 - Display daily/weekly outlook: Strong Uptrend, Uptrend, Sideways, Downtrend, Strong Downtrend.
 - Show indicator status: RSI(14), MFI(14), MACD, VW SMA, EMA.
 - List support/resistance levels with trading recommendations.
- **Features:**
 - AI-driven insights (e.g., “Buy above R1 if volume confirms”).
 - Historical performance of recommendations.
- **Implementation:**
 - Compute analysis with Polars, render with React.
 - Use AI agent for recommendations.

3.2.8. News and Events

- **Functionality:**
 - Display latest news for selected scrips.
 - Show upcoming events (AGM, dividends, earnings) within 10 days.
 - Link to full articles where available.
- **Features:**
 - Filter by relevance or date.
 - Highlight high-impact events.
- **Implementation:**

- Placeholder for future API integration (e.g., Reuters).
- Mock data for initial phase.

3.3. Data Management

Description: Manages ingestion, processing, storage, and visualization of market data.

3.3.1. Data Ingestion

- **Functionality:**
 - Fetch real-time OHLCV data via IBKR WebSocket.
 - Retrieve historical data (up to 5 years) for backtesting.
 - Poll IBKR every 5-10 seconds, respecting rate limits.
- **Features:**
 - Cache data in Redis for low-latency access.
 - Handle API disconnections with automatic reconnection.
- **Implementation:**
 - `ib_insync` for IBKR integration.
 - Redis for caching with TTL.

3.3.2. Data Processing

- **Functionality:**
 - Normalize timestamps to UTC and align asset identifiers.
 - Clean data: Remove outliers, fill missing values with interpolation.
 - Compute indicators (VW SMA, MFI, ATR, etc.) in parallel.
- **Features:**
 - Validate data integrity (e.g., non-negative volumes).
 - Log processing errors for debugging.
- **Implementation:**
 - Polars for parallel processing.
 - Custom validation scripts.

3.3.3. Data Storage

- **Functionality:**
 - Store time-series data (market, trades, positions) in DuckDB with Parquet.
 - Partition by date and asset for efficient queries.
 - Retain data for 7 years per regulatory requirements.
- **Features:**
 - Daily backups to external storage.
 - Compress data to optimize storage.
- **Implementation:**
 - DuckDB with Parquet export.
 - Cron jobs for backups.

3.3.4. Data Visualization

- **Functionality:**
 - Render price, indicator, and performance charts.
 - Support multi-timeframe analysis (1m to 1w).
 - Export visualizations as PNG or CSV.
- **Features:**
 - Interactive zooming and panning.
 - Customizable chart styles.
- **Implementation:**
 - Plotly.js for frontend rendering.
 - Polars for data aggregation.

3.4. Order Execution

Description: Manages the trade lifecycle in the IBKR paper trading environment.

3.4.1. Order Types

- **Functionality:**
 - Support market, limit, stop, and bracket orders.
 - Bracket orders include entry, take-profit, and stop-loss.
- **Features:**

- Validate order parameters (price, quantity).
- Support partial fills.

3.4.2. Order Placement

- **Functionality:**
 - Submit orders via `ib_insync` with validation.
 - Calculate position size based on risk parameters.
 - Confirm order submission with user notification.
- **Features:**
 - Preview order impact (e.g., margin, fees).
 - Log order details for auditing.

3.4.3. Order Monitoring

- **Functionality:**
 - Track order status in real time (pending, filled, canceled).
 - Update portfolio and P&L upon execution.
 - Notify users of status changes.
- **Features:**
 - Display order history with filters.
 - Allow cancellation of pending orders.

3.4.4. Error Handling

- **Functionality:**
 - Handle API errors (e.g., rate limits, connectivity issues).
 - Retry failed orders up to 3 times with exponential backoff.
 - Log errors with timestamps and context.
- **Features:**
 - Alert administrators for critical failures.
 - Fallback to manual trading if API is unavailable.

3.5. Risk Management

Description: Enforces controls to protect capital and ensure sustainable trading.

3.5.1. Position Sizing

- **Functionality:**
 - Limit risk to 1-2% per trade based on account balance and ATR.
 - Adjust for lot size and asset type.
- **Formula:**
 - Trend-Following:

$$\left[\begin{aligned} \text{Position Size} &= \frac{\text{Account Balance} \times 0.01}{2 \times \text{21-day VW ATR} \times \text{Lot Size}} \end{aligned} \right]$$
 - Support and Resistance:

$$\left[\begin{aligned} \text{Position Size} &= \frac{\text{Account Balance} \times \text{Risk \%}}{|\text{Entry Price} - \text{Stop Loss Price}|} \end{aligned} \right]$$

3.5.2. Stop-Loss and Take-Profit

- **Functionality:**
 - Set stop-loss and take-profit at trade entry.
 - Trend-Following: $2 \times 21\text{-day VW ATR}$.
 - Support and Resistance: Based on S1/R1 or 1-2% from entry.
- **Features:**
 - Place orders with IBKR API.
 - Validate stop-loss/take-profit levels.

3.5.3. Trailing Stop

- **Functionality:**
 - Trend-Following: $1.5 \times 21\text{-day VW ATR}$.
 - Support and Resistance: 2-3% or $2 \times 14\text{-day ATR}$.
 - Update dynamically via IBKR API.

- **Features:**
 - Prevent premature exits during volatility.
 - Log trailing stop updates.

3.5.4. Portfolio Risk

- **Functionality:**
 - Limit total open risk to 5-10% of account balance.
 - Monitor correlations to avoid concentrated exposure.
 - Diversify across assets and sectors.
- **Features:**
 - Alert if risk exceeds thresholds.
 - Suggest rebalancing actions.

3.5.5. Drawdown Limits

- **Functionality:**
 - Reduce risk to 0.5% per trade if drawdown > 20%.
 - Halt trading if drawdown > 30% until manual review.
- **Features:**
 - Track drawdown in real time.
 - Notify traders of limit breaches.

3.5.6. Circuit Breakers

- **Functionality:**
 - Halt trading during extreme volatility (e.g., price change > 5% in 5 minutes).
 - Pause if system detects anomalies (e.g., API errors, data inconsistencies).
- **Features:**
 - Send alerts to administrators.
 - Resume after manual approval.

3.5.7. Currency Handling

- **Functionality:**
 - Convert risk calculations to GBP using IBKR exchange rates.
 - Handle multi-currency assets transparently.
- **Features:**
 - Cache exchange rates for performance.
 - Log conversions for auditing.

3.6. AI Agents Integration

Description: Modular AI agents automate trading tasks under human oversight.

3.6.1. Trade Execution Agent

- **Functionality:**
 - Execute trades based on strategy signals.
 - Optimize order timing and price.
- **Features:**
 - Use reinforcement learning for execution optimization.
 - Log decisions for transparency.

3.6.2. Risk Management Agent

- **Functionality:**
 - Enforce position sizing, exposure limits, and circuit breakers.
 - Adjust risk dynamically based on market conditions.
- **Features:**
 - Monitor portfolio metrics in real time.
 - Suggest risk mitigation actions.

3.6.3. Market Analysis Agent

- **Functionality:**
 - Analyze trends, volatility, and momentum.
 - Generate trading recommendations.
- **Features:**
 - Use pre-trained models (e.g., LSTM for price prediction).
 - Provide explainability with SHAP.

3.6.4. IBKR API Agent

- **Functionality:**
 - Manage API interactions for data and orders.
 - Handle rate limits and errors.
- **Features:**
 - Optimize API calls with caching.
 - Log API performance metrics.

3.6.5. Transparency and Oversight

- **Functionality:**
 - Log all agent decisions with timestamps and rationale.
 - Require human approval for critical actions.
 - Allow pausing/overriding agents via dashboard.
- **Features:**
 - Audit trails for compliance.
 - Explainable AI for trader trust.

3.7. User Interface

Description: A responsive web interface for traders.

3.7.1. Frontend Components

- **Functionality:**
 - Dropdown/input for scrip selection.
 - Real-time price/volume tables.
 - Candlestick/line charts with Plotly.js.
 - Portfolio/order status tables.
 - Trading form with validation.
 - Watchlist with highlights.
 - Technical analysis panel.
 - News/events section.
- **Implementation:**
 - React 18 with Tailwind CSS.

3.7.2. Interactivity

- **Functionality:**
 - Update data via WebSocket.
 - Handle user inputs (e.g., order submission) with callbacks.
 - Provide tooltips and help icons.
- **Features:**
 - Smooth transitions for chart updates.
 - Undo/redo for critical actions.

3.7.3. Responsiveness

- **Functionality:**
 - Optimize for desktop, tablet, and mobile (iPhone, Android).
 - Adjust layouts dynamically.
- **Features:**
 - Test on multiple screen sizes.
 - Support touch gestures for mobile.

3.7.4. Accessibility

- **Functionality:**
 - Comply with WCAG 2.1 (e.g., color contrast, keyboard navigation).
 - Provide alt text for charts.
- **Features:**
 - Screen reader compatibility.
 - High-contrast mode.

3.8. Reporting and Analytics

Description: Generates reports for performance, analytics, and compliance.

3.8.1. Trade Performance

- **Functionality:**

- Metrics: P&L, win rate, Sharpe ratio, max drawdown, annualized return.
 - Visualize equity curves and drawdowns.
 - **Features:**
 - Filter by strategy, asset, or timeframe.
 - Compare with benchmarks.
 - 3.8.2. Portfolio Analytics**
 - **Functionality:**
 - Exposure, correlations, VaR, expected shortfall.
 - Diversification analysis.
 - **Features:**
 - Highlight high-risk positions.
 - Suggest rebalancing.
 - 3.8.3. Backtesting Reports**
 - **Functionality:**
 - Compare strategy performance on historical data.
 - Include sensitivity analysis for parameters.
 - **Features:**
 - Visualize backtest results.
 - Export as CSV/PDF.
 - 3.8.4. Compliance Reports**
 - **Functionality:**
 - Transaction logs with timestamps and user IDs.
 - Audit trails for regulatory reporting.
 - **Features:**
 - Filter by date or user.
 - Ensure immutability.
 - 3.8.5. Export Capabilities**
 - **Functionality:**
 - Export reports as CSV, PDF, or PNG.
 - Schedule automated report generation.
 - **Features:**
 - Secure export with user authentication.
 - Compress large files.
-

4. Non-Functional Requirements

4.1. Performance

- End-to-end latency: <100ms for data to order submission.
- Throughput: 10,000 market updates/second, 1,000 trades/minute.
- Indicator calculation: <1s for 100 scrips.
- Dashboard refresh: Every 5-10s without lag.
- Backtesting: 5 years of data for 10 scrips in <10min.

4.2. Scalability

- Vertical: Leverage Lenovo Legion Pro 5's GPU/CPU.
- Horizontal: Support microservices for future cloud.
- Data: Handle 1TB of historical data.
- Users: Support 10 concurrent dashboard users.

4.3. Security

- Authentication: OAuth2 with JWT and MFA.
- Data Protection: TLS 1.3, AES-256 encryption.
- Credentials: Store in encrypted vault.
- API Security: Rate limiting, CSRF tokens, input validation.
- Agents: Sandbox in isolated containers.

4.4. Reliability and Fault Tolerance

- Uptime: 99.9% excluding maintenance.
- Error Handling: Automatic API reconnection.

- Data Integrity: Validate market data.
- Circuit Breakers: Halt trading during anomalies.
- Backup: Daily with RTO <1 hour.

4.5. Usability

- Intuitive navigation: <5min training.
- UI response: <1s for actions.
- Accessibility: WCAG 2.1 compliance.
- Documentation: User guides in Confluence.

4.6. Compliance

- Adhere to SEC, GDPR, MiFID II, SEBI.
- Log all trades/orders immutably.
- Retain transaction logs for 7 years.
- Monitor for market abuse in real time.

5. Integration Requirements

5.1. Interactive Brokers Paper Trading API

- **Connection:** Host: 127.0.0.1, Port: 7497, Client ID: 0, Account: DUK221396.
- **Data:** Real-time/historical OHLCV via WebSocket.
- **Orders:** Market, limit, stop, bracket; monitor status.
- **Rate Limits:** 50 requests/s (data), 100 messages/s (orders).
- **Security:** Store credentials in encrypted vault.

5.2. External Data Sources

- Placeholder for news feeds (Reuters, Bloomberg).
- Future sentiment and economic data integration.
- Use REST/WebSocket for real-time, batch for historical.

5.3. Internal Systems

- Sync with portfolio tools for benchmarking.
- Export logs to compliance systems.
- Store data in internal warehouse via REST APIs.

6. Assumptions and Dependencies

6.1. Assumptions Log

ID	Description	Impact if Invalid	Validation Plan
A001	IBKR API is reliable	Delayed/inaccurate trades	Test API connectivity
A002	Traders know technical analysis	Increased training	Conduct training sessions
A003	Hardware meets requirements	Performance issues	Benchmark hardware
A004	Minimal paper trading regulations	Compliance delays	Consult Compliance Officer

6.2. Dependencies

- IBKR Paper Trading API availability.
- Polars, DuckDB, and ib_insync libraries.
- Lenovo Legion Pro 5 hardware.
- Development team expertise in Python/React.

7. Missing Information and Best Practices

7.1. Missing Information

- Exact VW indicator formulas (e.g., VW MFI weighting).
- Detailed trader personas (experience, preferences).
- Specific regulatory requirements for paper trading.
- Performance benchmarks for similar systems.
- Preferred news feed providers.

7.2. Best Practices Applied

- **Indicator Formulas:** Used standard VW calculations (e.g., $VW\ SMA = \frac{\sum(Price \times Volume)}{\sum Volume}$). **Reason:** Industry-standard, ensures accuracy.

- **User Personas:** Assumed intermediate trader knowledge. **Reason:** Typical for hedge funds, focuses usability.
- **Regulatory Compliance:** Added audit trails and logging. **Reason:** Prepares for live trading, covers common regulations.
- **Performance:** Targeted <100ms latency, 10,000 updates/s. **Reason:** Aligns with high-frequency trading standards.
- **News Feeds:** Planned for Reuters/Bloomberg integration. **Reason:** Reliable sources for financial data.

8. Glossary

- **ATS:** Algorithmic Trading System.
- **IBKR:** Interactive Brokers.
- **VW SMA:** Volume-Weighted Simple Moving Average.
- **MFI:** Money Flow Index.
- **ATR:** Average True Range.
- **Polars:** Data processing library.
- **DuckDB:** Analytical database.
- **FastAPI:** API framework.
- **ib_insync:** IBKR API library.

9. Appendices

9.1. References

- IBKR API: <https://interactivebrokers.github.io/tws-api/>
- Polars: <https://pola.rs/>
- DuckDB: <https://duckdb.org/>
- FastAPI: <https://fastapi.tiangolo.com/>
- React: <https://reactjs.org/>
- Plotly.js: <https://plotly.com/javascript/>
- SEC: <https://www.sec.gov/>
- GDPR: <https://gdpr.eu/>

9.2. Sample Data Formats

OHLCV Data:

```
{
  "ticker": "AAPL",
  "exchange": "NASDAQ",
  "timestamp": "2025-04-24T10:00:00Z",
  "open": 150.25,
  "high": 151.10,
  "low": 149.80,
  "close": 150.90,
  "volume": 1000000
}
```

Trade Log:

```
{
  "trade_id": "T12345",
  "timestamp": "2025-04-24T10:01:00Z",
  "ticker": "AAPL",
  "strategy": "Trend-Following",
  "action": "BUY",
  "quantity": 100,
  "price": 150.90,
  "status": "FILLED"
}
```

9.3. Mockups and Wireframes

- **Dashboard Layout:** Grid with scrip selector, chart, portfolio table, trading panel.

- **Chart View:** Candlestick with indicator overlays, zoom controls.
- **Trading Panel:** Form with ticker, quantity, price, order type dropdown.